

phase_04_GAP_AUDIT

Phase 4 — Virtual Testing Matrix: GAP AUDIT

Honest completion: ~30% complete. Pure-Go DST/chaos/device-profile/snapshot/wasm primitives + a helix-test CLI are real and tested, but the roadmap’s defining infrastructure (Firecracker microVMs, K3s, Rust turmoil, Elixir/OTP controller, Phoenix dashboard, 25+ fault types, HelixQA pipeline) is absent or only protocol-shaped. The matrix cannot yet “simulate all 8 tiers” or gate CI as specified.

Audited 2026-06-01 against actual code under pkg/, internal/, cmd/, containers/. All cited tests were executed (go test ./pkg/testing/... ./pkg/wasm/... ./cmd/helix-test/... → all ok).

3-Axis Package Status (Refreshed 2026-06-04, HXC-939)

Why this section exists: “exists” ≠ “used”. A package can be **implemented** and **tested** yet never reached from any binary (**orphaned**). wired is **measured** via go list -deps ./cmd/... | grep -Fx <module-path>/<pkg> (module = github.com/HelixDevelopment/helix_cluster), not assumed. **“Completed (registry) ≠ wired”** — Completed only means source+tests exist; it does NOT prove a shipped binary reaches it.

Package	implemented	wired (reachable from cmd/)	tested
pkg/testing/dst	yes	yes	yes
pkg/testing/chaos	yes	yes	yes
pkg/testing/device	yes	yes	yes
pkg/testing/snapshot	yes	yes	yes

Package	implemented	wired (reachable from cmd/)	tested
pkg/wasm	yes	NO (orphaned)	yes
cmd/helix-test	yes	yes	yes
cmd/helix-snapshot	yes	yes	yes
cmd/helix-testd	NO (absent)	n/a	n/a
containers/pkg/vm (QEMU, Phase-1 carryover)	yes	NO (orphaned)	yes

Orphan callouts (implemented+tested but NOT reachable from any cmd/binary): pkg/wasm, containers/pkg/vm. Note cmd/helix-snapshot now **exists and is built** (the old text marked it MISSING — that is stale; the standalone binary is present). cmd/helix-testd is still absent.

Deliverable Status

Deliverable (roadmap §)	Status	Evidence (file:line)
pkg/testing/dst — virtual time, seeded PRNG, event queue (§5.1, P0#4)	PARTIAL	pkg/testing/dst/engine.go:66–203; engine_test.go
Interface swapping (Net2/Sim2) (P0#5)	MISSING	grep: no Sim2/Net2/SimNet any .go
BUGGIFY macro system (P1#10)	MISSING	grep: no BUGGIFY in repo
pkg/testing/chaos — fault injection (§5.1, P0#6)	PARTIAL	pkg/testing/chaos/fault.go:17–296; fault_test.go

Deliverable (roadmap §)	Status	Evidence (file:line)
25+ fault injection types (§2, P1#11)	MISSING	only 5 structs in fault.go:25,54,106,163
pkg/testing/device — profile registry (§5.1, P0#3)	PARTIAL	pkg/testing/device/ profile.go:55–182; profile_test.go
pkg/testing/snapshot — golden snapshot mgmt (§5.1)	DONE	pkg/testing/snapshot/ manager.go:14–114; manager_test.go
pkg/wasm — Wasmtime host + plugin (§5.1, P1#14, §4.7)	PARTIAL →strong	pkg/wasm/host.go:28–12 plugin.go:35–103; plugin_test.go:13–56 re Wat2Wasm modules
cmd/helix-test CLI (§5.3, P0)	DONE	cmd/helix-test/ main.go:44–247; main_test.go (294 lines)
cmd/helix-testd daemon (OTP controller node) (§5.3, P0)	MISSING	ls cmd → absent
cmd/helix-snapshot CLI (§5.3, P1)	MISSING	ls cmd → absent
K3s foundation + RuntimeClasses + golden image pipeline (§3 4.1, P0#1-2)	MISSING / DEFERRED	grep: no k3s/RuntimeClass .go
Firecracker microVM T1–T3 (28ms boot) (§4.2, P0#2)	MISSING / DEFERRED	grep: no firecracker in .g
QEMU/KVM T4–T6 full-system emulation (§4.2, P1#8)	PARTIAL (Phase-1 carryover)	containers/pkg/vm/ qemu.go:74–188, matrix. (429 ln); qemu_test.go

Deliverable (roadmap §)	Status	Evidence (file:line)
Docker T7–T8 protocol stubs (iOS/HarmonyOS) (§4.2, P1#9)	MISSING	containers/pkg/vm/ios,macos dirs exist but no T8 binfmt path
Elixir/OTP controller: SessionManager/DevicePool/TestRunner/SnapshotManager (§5.2, P0#7)	MISSING / DEFERRED	no *.ex/mix.exs in repo
Phoenix LiveView dashboard (§5.2, P1#12)	MISSING / DEFERRED	no Elixir/Phoenix sources
internal/testing/{controller,dashboard,faults,workloads} (§5.2)	MISSING	internal/testing/ does not exist
HelixQA integration: invariant detection, Welch's t-test regression, CI gate (§3 4.6, P1#13)	MISSING	grep Welch/invariant/regression → only unrelated hits
Success-criteria benchmarks (boot 28ms, 5000 VMs/host, 10:1 compression, <5µs spawn, >80% cov) (§9)	MISSING	no benchmark suites for these KPIs

TOP IMPLEMENTABLE GAPS (Go, no new infra)

- 1. Expand chaos fault library toward 25+ — pkg/testing/chaos/.** Add pure-Go in-sim faults implementing the existing Fault interface: ClockSkew, DiskFill, MessageReorder, MessageDuplication, ByzantineCorruption, DNSFailure, CertExpiry, SlowDisk, Bitflip, ConnectionReset. Each holds parameters + applied state and exposes a query method (e.g. ShouldCorrupt(), Skew()) the DST engine consults. Keeps the existing apply/restore symmetry and ChaosRunner. *Acceptance test (mutation-pairable):* seed a fixed RNG, assert ClockSkew.Skew() returns the exact deterministic offset sequence and is zero before Apply()/after Restore(); mutant that drops the applied guard or mis-signs the offset fails.
- 2. Wire DST into a real distributed pkg via a sim transport seam — pkg/testing/dst/ + pkg/swim/.** Define a Transport interface (Send(from,to string, msg []byte)) with a production net impl and a SimTransport backed by Engine.SendMessage/handlers, then run pkg/swim membership over it. This realizes “same code in prod and sim” (P0#5) without external infra. *Acceptance test:* run SWIM over SimTransport with seed S, partition two nodes via a chaos.NetworkPartition consulted by the transport, assert the partitioned node is marked suspect/dead deterministically and identically across two runs of seed S; mutant that ignores the partition predicate keeps the node alive → fails.

3. **Add a BUGGIFY probabilistic fault hook — pkg/testing/dst/**. Implement `func (eng *Engine) Buggify(p float64) bool` driven by the engine RNG, plus a registration list so enabled BUGGIFY sites are reproducible per seed and can be globally toggled. *Acceptance test*: with a fixed seed, assert the exact boolean sequence for `Buggify(0.5)` over N calls and that disabling BUGGIFY makes it always false; mutant using a fresh/global rand instead of `eng.rng` breaks reproducibility → fails.
4. **Statistical regression detector — pkg/testing/regression/ (new) feeding HelixQA**. Implement Welch's t-test (`func Welch(a, b []float64) (t, dfWelch, p float64)`) and a Detector that flags a regression when $p < \alpha$ and mean degrades beyond a threshold, returning a structured result the challenge pipeline can publish. Pure numeric, no infra. *Acceptance test*: feed two known samples with a published expected t-stat/p-value (textbook fixture), assert within epsilon; assert identical distributions yield “no regression”; mutant swapping pooled-variance for Welch's separate-variance denominator fails the fixture.
5. **Device provisioning lifecycle abstraction — pkg/testing/device/**. Add a Provisioner interface (`Provision(*Profile) (Instance, error)`, `Instance.Boot/Stop/Status`) with a deterministic FakeProvisioner (state machine: provisioned→booting→ready→stopped, timings from the profile). Lets the controller/CLI exercise lifecycle logic without Firecracker/QEMU. *Acceptance test*: provision T3, assert state transitions and that Boot is rejected from stopped; assert profile boot-time is honored against a virtual clock; mutant that allows boot-from-stopped or skips a transition fails.
6. **cmd/helix-snapshot standalone CLI — cmd/helix-snapshot/**. Promote the snapshot subcommand into its own P1 binary reusing `pkg/testing/snapshot`, with the same injectable `run(args, stdout, stderr) int` seam as `helix-test`. *Acceptance test*: `create→list→compare(match)→compare(mismatch→exit1)→delete` sequence via the run seam over a temp dir, asserting exit codes and stdout; mutant returning 0 on mismatch fails.
7. **Capability sandbox + spawn benchmark for wasm — pkg/wasm/**. Add an allow-list Capabilities struct gating which WASI features `Instantiate` enables (deny FS/clock/env by default), and a `Benchmark/testing.B` measuring host+instantiate latency. *Acceptance test*: a module importing a denied host function fails to instantiate with a typed error; an allowed one succeeds; mutant that ignores the capability set (always full WASI) lets the denied module run → fails.

DEFERRED (need external infra / non-Go)

- **K3s cluster + RuntimeClasses + golden-image pipeline** — needs a real Kubernetes cluster and node runtimes; cannot be unit-validated in-repo.
- **Firecracker microVMs (T1–T3, 28ms boot, 5000 VMs/host)** — requires KVM-enabled Linux host + firecracker binary + snapshot files.
- **QEMU/KVM T4–T6 boot validation** — provisioner code exists (`containers/pkg/vm/qemu.go`) but live booting needs QEMU/KVM hardware; only mock-level tests run in CI.

- **Cuttlefish (T5 Android) / iOS (T7) / HarmonyOS (T8) real device sim** — Android virtualization, Apple-licensed virtualization (\$9,995 tier per §8), and HarmonyOS images; protocol-stub Go work is in gap #1-style scope but real-device fidelity is hardware-bound.
- **Elixir/OTP Chaos & Testing Controller + Phoenix LiveView dashboard** — non-Go stack (Elixir/BEAM); out of this Go-only audit's implementable set.
- **Rust turmoil DST core** — non-Go; the Go `dst.Engine` is the in-repo substitute.

Deferred count: 6 areas (all require external infra, hardware, or non-Go runtimes; reasons above).